AI Log - Weekly BTC/Kalshi Build Script Debugging

Representative conversation: fixing a Polars schema mismatch and building weekly contract behavior files for the dashboard.

Context

The user was running a weekly build script to combine Kalshi analysis, labels, and futures windows. The goal was to create weekly files that Streamlit could load without trying to visualize more than one week at a time.

Representative error

The script failed while joining labels to futures window behavior because the join keys had the same name but different timestamp precisions.

```
polars.exceptions.SchemaError: datatypes of join keys don't match - `window_start`:
datetime[μs, UTC] on left does not match `window_start`: datetime[ns, UTC] on right
```

Representative exchange

User: "whats the revised script"

Assistant: Rebuilt the full script so timestamp columns were normalized before joins, especially window_start and related datetime columns. The assistant focused on making the weekly build deterministic and dashboard-ready rather than doing expensive transformations inside Streamlit.

Design decisions

The assistant and user agreed that the dashboard should never attempt to load or visualize an open-ended date range. Instead, the preprocessing pipeline should produce weekly files and the Streamlit app should use a week selector. This was both a performance decision and a rubric alignment decision: the app needed working filters and interactivity but should avoid crashing from too much data.

Additional data discovered

The user later found Kalshi 1-second and decision-time data files, including kalshi_btc15m_grid_1s.parquet, kalshi_btc15m_observed_1s.parquet, kalshi_model_1s_combined.parquet, and kalshi_contract_decision_times.parquet. These files became the basis for contract evolution, sensitivity, threshold-risk, and decision-time visualizations.

Outcome

The weekly-file design became the backbone of the implementation. Instead of recomputing everything interactively, the app loads precomputed weekly files such as contract_summary.parquet, contract_evolution_1s.parquet, contract_plot_sample.parquet, sensitivity_bins.parquet, threshold_heatmap.parquet, and contract_decision_times.parquet.